

Fast Matrix Multiplication

Strassen's Algorithm: An Empirical Study (A)

Rodrigo Uzal Sequeros | Student ID: 26135071 | September 27, 2026

1 Implementation

1.1 Algorithm Overview

In this assignment, an implementation of Strassen's algorithm is tested against regular matrix multiplication in a range of different scenarios to determine which is more efficient under what circumstances. Let us first define the algorithm for each multiplication form.

1.1.1 Standard Matrix Multiplication

Let $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$, where the number of columns of A equals the number of rows of B . Their product is the matrix $C = AB \in \mathbb{R}^{m \times n}$ whose entries are given by

$$c_{ij} = \sum_{l=1}^k a_{il} b_{lj}, \quad 1 \leq i \leq m, \quad 1 \leq j \leq n. \quad (1)$$

That is, the entry c_{ij} is the inner product of the i -th row of A with the j -th column of B .

Equation (1) translates directly into an algorithm: three nested loops over i , j and l that accumulate the sum for each entry of C . Computing one entry requires k multiplications and $k - 1$ additions, so in total there are required

$$mnk \text{ multiplications} \quad \text{and} \quad mn(k - 1) \text{ additions}, \quad (2)$$

that is $mn(2k - 1)$ arithmetic operations in total. In the square case $m = n = k$ this gives n^3 multiplications and $n^2(n - 1)$ additions, and hence a running time of $\Theta(n^3)$.

The order in which the three loops are nested does not affect the result, since each c_{ij} accumulates the same k terms regardless, but it does affect the order in which memory is accessed, which is why this implementation uses the **i-k-j** order (Section 1.2).

1.1.2 Strassen's Algorithm

Let $A, B \in \mathbb{R}^{n \times n}$ be square matrices whose size is a power of two, $n = 2^p$, and let $C = AB$. This assumption is made only to keep the exposition simple: any other size can be reduced to this case, as discussed in Section 1.3. Partition each matrix into four blocks of size $\frac{n}{2} \times \frac{n}{2}$,

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}, \quad C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}. \quad (3)$$

Block multiplication then gives

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{bmatrix}. \quad (4)$$

Applied recursively, (4) yields no improvement: it requires eight products of half-size matrices, so its cost satisfies $T(n) = 8T(n/2) + \Theta(n^2)$, which solves to $\Theta(n^3)$, the same order as the standard algorithm.

Strassen's algorithm instead computes the seven auxiliary products

$$\begin{aligned} M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}), \\ M_2 &= (A_{21} + A_{22}) B_{11}, \\ M_3 &= A_{11} (B_{12} - B_{22}), \\ M_4 &= A_{22} (B_{21} - B_{11}), \\ M_5 &= (A_{11} + A_{12}) B_{22}, \\ M_6 &= (A_{21} - A_{11})(B_{11} + B_{12}), \\ M_7 &= (A_{12} - A_{22})(B_{21} + B_{22}), \end{aligned} \tag{5}$$

each of which involves a single multiplication of $\frac{n}{2} \times \frac{n}{2}$ matrices, and recovers the blocks of C from them using only additions and subtractions:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 - M_2 + M_3 + M_6 \end{bmatrix}. \tag{6}$$

The correctness of (6) follows by substituting the definitions (5) and cancelling: for instance, in C_{11} the terms $A_{11}B_{22}$, $A_{22}B_{11}$, $A_{22}B_{22}$, $A_{22}B_{21}$ and $A_{12}B_{22}$ each occur once with a positive and once with a negative sign, leaving $A_{11}B_{11} + A_{12}B_{21}$ as required by (4).

The efficiency comes from the fact that the seven products are shared between the four blocks of C : five of them appear in two different blocks. Since each recursive call performs seven multiplications of half-size matrices, together with a constant number of matrix additions costing $\Theta(n^2)$, the running time satisfies

$$T(n) = 7T\left(\frac{n}{2}\right) + \Theta(n^2), \tag{7}$$

and because $n^{\log_2 7}$ dominates n^2 , the Master Theorem gives

$$T(n) = \Theta\left(n^{\log_2 7}\right) \approx \Theta(n^{2.807}). \tag{8}$$

The recursion is not carried down to 1×1 matrices. Below a cut-off size n_0 , which `strassen_mul` takes as a parameter, blocks are multiplied with the standard algorithm instead, because at small sizes the overhead of the block operations outweighs the saving in multiplications. The choice of this threshold is examined experimentally in Section 2.

1.2 Design Decisions

One of the main design decisions was to make use of two distinct programming languages. First, for the main algorithm and calculations, I used C++ as it is much more efficient for larger computations, as a consequence of it being a compiled language. This meant that I could handle test cases with a greater variety of variables. On the other hand, for plotting, I made use of the intuitive matplotlib library in Python. This has led to a hybrid project that makes use of the best properties of both worlds.

Another important design decision was defining the matrix structure as long 1D vectors, which led to a much more intuitive and efficient implementation of these mathematical objects. It did mean that for loop structures such as standard matrix multiplication, I followed the loop order `i-k-j` instead of `i-j-k`.

Finally, Strassen's algorithm requires constant block partitioning of matrices. This process is particularly costly for matrices whose size is not a power of two, $n \neq 2^p$, since in these cases the matrices have to be padded with zero entries or handled by peeling (Section 1.3).

1.3 Simplifications and Extensions

This implementation departs from the common form of the algorithm in five respects, each adopted deliberately.

- **Restriction to square matrices of size a power of two.** `strassen_mul` accepts only inputs with $n = 2^p$ and rejects any other size. The general case can be handled either by padding the operands with zero rows and columns up to the next power of two, or by peeling off the odd row and column at each level and correcting the result with a rank-one update. Padding is simple but may almost double the effective size, increasing the work by a factor of up to $2^{\log_2 7} = 7$ in the worst case, while peeling complicates every level of the recursion. Since the experiments reported here use sizes that are powers of two, neither mechanism would affect the measurements, and both would introduce an unnecessary cost into the comparison.
- **Truncated recursion.** The recursion is not carried down to 1×1 blocks. Below a threshold n_0 , the standard algorithm is called instead. At small enough sizes the eight block extractions, the eighteen matrix additions and the associated allocations outweigh the saving of one multiplication out of eight, so the asymptotically inferior algorithm is the faster one in that range. The effect of n_0 on the running time is measured in Section 2.
- **Blocks are copied rather than referenced.** Each recursive call extracts its eight sub-blocks into freshly allocated matrices, instead of representing a block as a pointer into the parent matrix together with a row stride. The asymptotic cost is unchanged, since copying a block is $\Theta(n^2)$ work, but the constant factor is not: the implementation allocates and copies at every level of the recursion, which is fair in exchange for a much simpler implementation.
- **Scope of the comparison.** Both algorithms operate on dense matrices of double-precision values, run on a single thread, and use no explicit vectorisation or specialized linear algebra library. The comparison is therefore between two implementations of comparable effort, and the resulting runtimes reflect general baseline performance rather than maximum possible speed.
- **Test matrices with entries in $[0, 1)$.** All experiments were run with dense random matrices with values drawn uniformly from $[0, 1)$. This prevents overflow and production of subnormal numbers, so the running times do not depend on the values, but they are the most favourable case for the accuracy of the standard algorithm: with nonnegative entries it never cancels, whereas Strassen’s algorithm subtracts blocks. Section 2.5 discusses the consequences for the accuracy results.

1.4 Code Structure

The repository is built around a single matrix type and two multiplication functions with the same interface (`strassen_mul` also takes the cut-off), so that either algorithm can be dropped into the same experiment without changing the surrounding code. A reader should start from `algorithms.hpp`, which declares both functions and nothing else, and then read `algorithms.cpp`, where `strassen_mul` validates its input and delegates the actual work to a private recursive function.

`matrix.hpp` Defines the `Matrix` type, stored as a single row-major vector, together with the element-wise helpers needed by Strassen’s implementation (addition, subtraction, block extraction and block insertion). Since they are short, they are defined in the header and are marked as `inline`.

`algorithms.hpp` Declares the public interface: `standard_mul` and `strassen_mul`. Both take two matrices and return their product, and `strassen_mul` also takes the cut-off n_0 as a third argument.

`algorithms.cpp` Implements both algorithms. They share a file because Strassen is not independent of the standard algorithm: below the recursion cut-off it calls it directly. Strassen is split in two parts. `strassen_mul` is a wrapper that checks the preconditions for multiplication and is called once per multiplication; the recursion itself lives in a private function that assumes its input is already valid, so no work is repeated at every level. The cut-off is a parameter of `strassen_mul` rather than a constant, so the experiments can vary it without recompiling.

`main.cpp` The experiment initialization. It generates the test matrices, verifies that both algorithms agree, measures their time and writes one row per measurement to a CSV file.

`plot_graph.py` Reads that CSV file and produces the figures in this report. The CSV is the only interface between the C++ and Python sections of the project.

`build.bat` Records the exact compiler and flags used for every measurement reported here.

2 Empirical Study

2.1 Objectives and Hypotheses

The experiments aim to determine under which circumstances, if any, Strassen’s algorithm is worth implementing and using in place of standard matrix multiplication. Four objectives are addressed. For each, a hypothesis was formulated before the main experiments were run, stating both the mechanism expected to govern the outcome and a quantitative prediction; H4 was partly informed by a preliminary correctness check (Section 2.2).

O1 Determine the size at which Strassen’s algorithm overtakes the standard one, for a fixed recursion cut-off.

H1 — The speedup ratio $t_{\text{std}}/t_{\text{str}}$ grows with n and eventually exceeds 1, since $\log_2 7 \approx 2.807 < 3$, but later than an operation count suggests. With a cut-off of 64, a single level of recursion at $n = 128$ performs $7 \cdot 2 \cdot 64^3 + 18 \cdot 64^2 \approx 3.74 \times 10^6$ floating-point operations, against $2 \cdot 128^3 \approx 4.19 \times 10^6$ for the standard algorithm, so counting operations alone predicts Strassen’s algorithm to be faster from $n = 128$. The count ignores memory traffic: every level copies eight blocks, performs eighteen matrix additions and allocates the corresponding temporaries, none of which the standard algorithm incurs. These operations are limited by memory bandwidth rather than by arithmetic, whereas the multiplications Strassen’s algorithm saves would have been executed by the fast, sequential **i-k-j** loop that also computes its base cases, so each saved multiplication is worth little compared with the work added. The crossover is therefore expected between $n = 256$ and $n = 1024$.

O2 Determine the cut-off n_0 that minimises the running time of Strassen’s algorithm, and whether it depends on the matrix size n .

H2 — Running time as a function of n_0 is U-shaped: small cut-offs pay the cost of the block operations on sub-problems too small to benefit from the reduced number of multiplications, while large cut-offs suppress the recursion almost entirely. The minimum is expected between $n_0 = 32$ and $n_0 = 128$, and to remain approximately constant as n varies.

O3 Estimate the empirical growth exponents of both algorithms and compare them with the theoretical $\Theta(n^3)$ and $\Theta(n^{\log_2 7})$.

H3 — The exponent fitted over the larger sizes (Section 2.2) is measurably smaller for Strassen’s algorithm than for the standard one, by at least 0.1: near 3.0 for the standard algorithm and near 2.85 for Strassen’s. Summed over the recursion, the block copies and additions contribute a $\Theta(n^{\log_2 7})$ term, so they do not alter Strassen’s asymptotic exponent. At the sizes tested, however, each doubling of n adds one level of recursion, so that $t(2n) = 7t(n) + \Theta(n^2)$: the ratio $t(2n)/t(n)$ exceeds 7, and the fitted exponent should lie slightly above $\log_2 7$, approaching it from above as n grows. Both estimates are also sensitive to the memory hierarchy, since cache misses may raise either measured exponent once the matrices no longer fit in cache; the standard algorithm may consequently exceed 3.0.

O4 Quantify the loss of accuracy incurred by Strassen’s algorithm relative to the standard one.

H4 — Let

$$E(n) = \frac{\|C_{\text{str}} - C_{\text{std}}\|_{\max}}{\|C_{\text{std}}\|_{\max}} \quad (9)$$

denote the relative discrepancy between the two results, where $\|M\|_{\max} = \max_{i,j} |m_{ij}|$ is the largest entry in absolute value. $E(n)$ is expected to grow with n , roughly doubling each time n doubles, while remaining below 10^{-12} at the sizes tested. Strassen’s algorithm forms sums and differences of blocks before multiplying, so intermediate quantities may exceed the magnitude of the final entries and the subsequent cancellation loses precision. It therefore satisfies only a norm-wise error bound, rather than the element-wise bound of the standard algorithm, and its accuracy degrades faster with n .

2.2 Methodology

Both algorithms are written in C++17 and were compiled with MSVC at /O2. Everything runs on a single thread. Strassen’s cut-off is passed to `strassen_mul` at run time, so one executable covers every configuration.

The test matrices are square, and n is always a power of two (Section 1.3). Their entries are drawn uniformly from $[0, 1)$ by `std::mt19937`, seeded with $10\,000r + n$ for repetition r and size n . Each repetition therefore gets its own matrices, and any of them can be regenerated exactly. Values in this range never produce subnormal numbers, which is the main situation in which the speed of a floating-point operation depends on its operands. The choice matters for accuracy, then, but not for the timings (Section 2.5).

Table 1 lists what was measured. Because n halves at every level of recursion, a cut-off that is not a power of two behaves exactly like the power of two below it, so only powers of two are worth testing, and a cut-off n_0 means $\log_2(n/n_0)$ levels. Setting $n_0 = n$ switches the recursion off. `strassen_mul` then simply calls the standard algorithm, and the two times should match, which gives a built-in control. Configurations needed by more than one objective were measured only once. The largest size, $n = 4096$, ran as a separate pass because of how long it takes.

Only the call to the multiplication function is timed, with the C++ clock `steady_clock`; generating the matrices, checking the result and writing it out all happen outside the timer. One untimed multiplication runs before the first measurement. The repetitions form the outer loop and the configurations the inner one, so a passing slowdown hits one repetition of everything rather than the last few configurations. Each time is reported as the median over the repetitions, with the fastest and slowest as error bars. Speedups and relative times are first computed within a repetition, where both algorithms multiplied the same matrices moments apart, and only then summarised by their median.

Within each repetition and size, the standard algorithm always goes first, and its result serves as the reference. After every Strassen call, outside the timed region, the program records $\|C_{\text{str}} -$

Table 1: Configurations measured, five times each in the main run and three times at $n = 4096$.

Objectives	Algorithm	Sizes n	Cut-offs n_0
O1, O3, O4	standard	64, 128, ..., 2048	—
O1, O3, O4	Strassen	64, 128, ..., 2048	64
O2	Strassen	512	8, 16, 32, 128, 256, 512
O2	Strassen	1024	8, 16, 32, 128, ..., 1024
O2	Strassen	2048	8, 16, 32, 128, ..., 2048
O1, O3, O4	both	4096	64 (Strassen)

Table 2: Median running times at $n_0 = 64$ and the speedup $t_{\text{std}}/t_{\text{str}}$, with its range over the repetitions. The last column states whether Strassen’s algorithm was faster in every repetition, slower in every repetition, or neither.

n	t_{std} (s)	t_{str} (s)	speedup	range	Strassen
64	0.000296	0.000343	0.85	0.45–1.81	mixed
128	0.00290	0.00491	0.72	0.44–0.92	slower
256	0.0230	0.0196	1.11	1.07–1.21	faster
512	0.161	0.119	1.34	1.17–1.40	faster
1024	1.23	0.849	1.58	1.42–1.64	faster
2048	10.7	5.99	1.79	1.77–1.97	faster
4096	87.6	43.2	2.00	1.99–2.03	faster

$C_{\text{std}}\|_{\text{max}}$ and $\|C_{\text{std}}\|_{\text{max}}$. Their quotient is E in (9), and the run stops if $E > 10^{-9}$. Because this check reads every entry of the result, the compiler cannot optimise the multiplication away either. Before the main experiments, a quick correctness check compared the two algorithms at $n = 256, 512$ and 1024 , and its results partly shaped H4.

The analysis rules were set before looking at the main results. For O1, the crossover is the smallest size from which Strassen’s algorithm wins in every repetition, both at that size and at every larger one. Sizes with $n \leq n_0$ are left out, since there both algorithms run identical code. The point where the median speedup crosses 1 is reported too, interpolated in $\log_2 n$. In O2, any cut-off within 5% of the fastest counts as equally good. The exponents for O3 are least-squares slopes of $\log t$ against $\log n$ over $n \geq 512$, and the local exponents between consecutive sizes, $\log_2(t(2n)/t(n))$, show what a single slope hides.

2.3 Results

All of Strassen’s results agreed with the standard ones within the tolerance. The control runs behaved as they should: at $n = 512, 1024$ and 2048 , Strassen’s algorithm at $n_0 = n$ took a median of 0.96, 1.02 and 1.00 times as long as the standard algorithm. At $n = 64$ the same ratio is 1.17, even though both measurements run identical code, and single runs of that computation took anywhere from 0.26 to 0.58 ms. Anything shorter than a few milliseconds is noisy (Section 2.5).

Figures 1 and 2 show the running times and the speedup at $n_0 = 64$, and Table 2 gives the numbers. At $n = 128$ Strassen’s algorithm lost in every repetition, taking a median of 1.39 times as long as the standard one. From $n = 256$ upwards it won in every repetition. The median speedup crosses 1 at $n \approx 210$ and keeps growing: 1.34 at $n = 512$, 1.79 at $n = 2048$ and 2.00 at $n = 4096$, where Strassen’s algorithm needs 43 s against 88 s. The first size tested at which it wins, $n = 256$, is the lower end of the range H1 predicted, and the interpolated crossing falls just below it. H1 is supported, though only at the lower edge of its range.

Figure 3 and Table 3 give Strassen’s running time as a fraction of the standard algorithm’s at the same size. The curve has the same U shape at all three sizes, with its minimum at $n_0 = 32$. There Strassen’s algorithm needs 0.73, 0.62 and 0.55 of the standard algorithm’s time

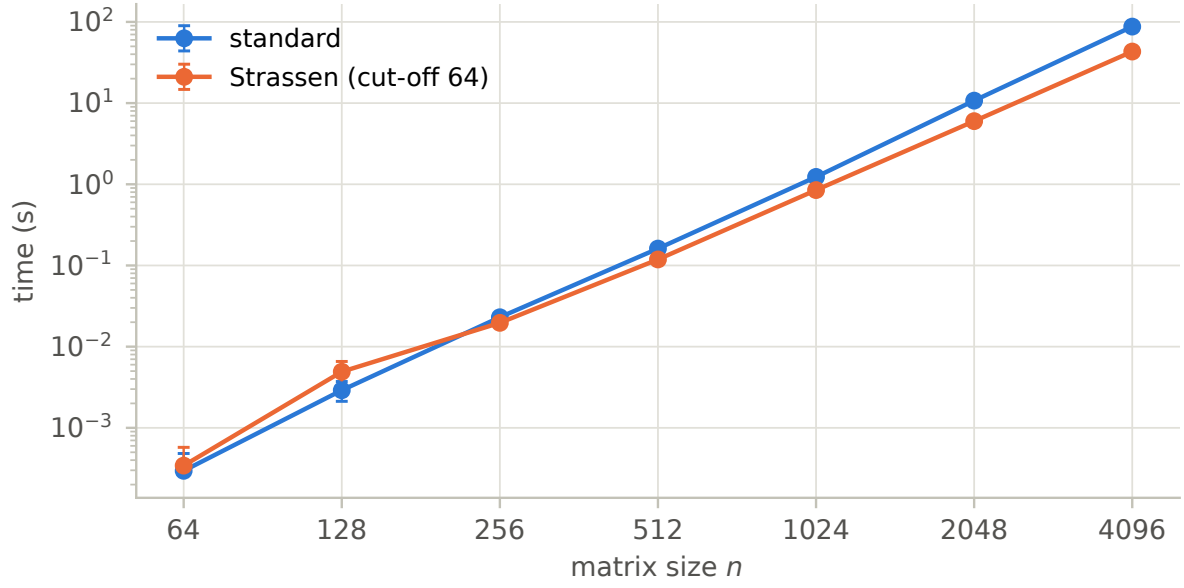


Figure 1: Running time of both algorithms at cut-off $n_0 = 64$: median of five repetitions (three at $n = 4096$), with error bars from the fastest to the slowest.

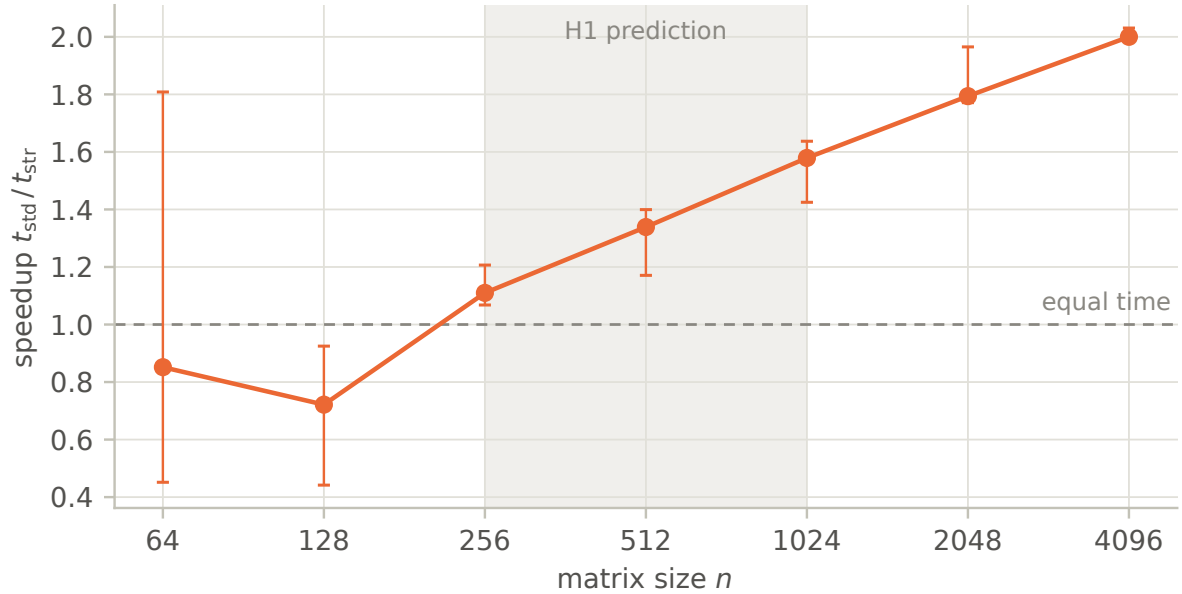


Figure 2: Speedup $t_{\text{std}}/t_{\text{str}}$ at $n_0 = 64$, computed within each repetition. The shaded band is the crossover range predicted by H1. At $n = 64$ both algorithms execute the same code.

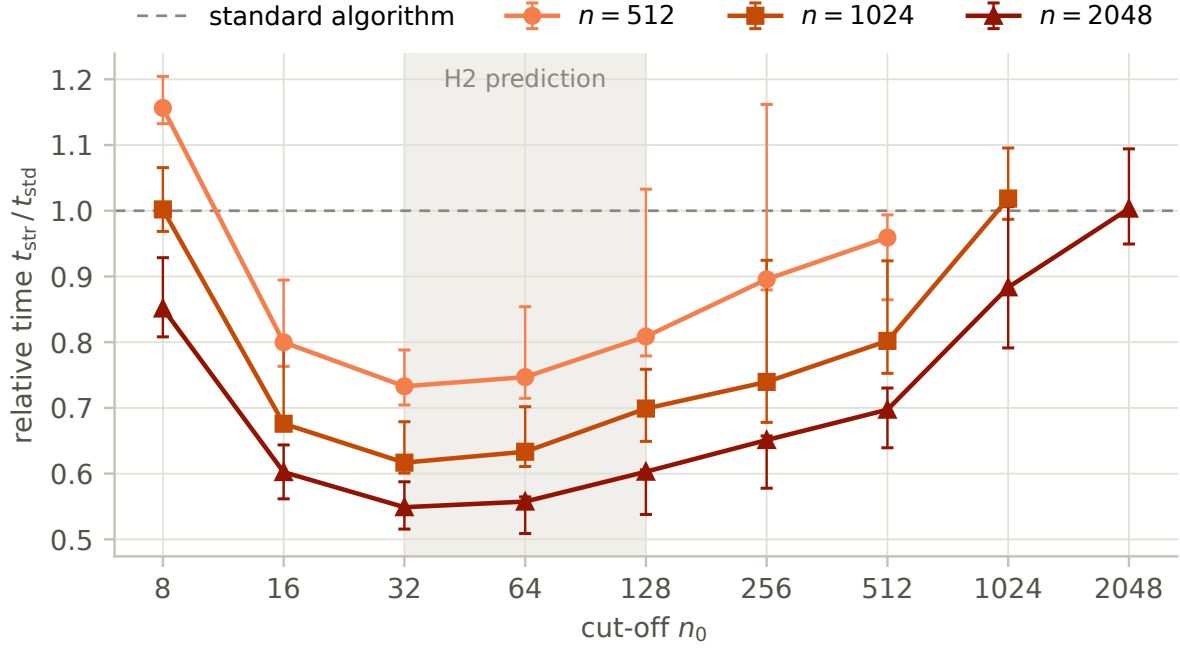


Figure 3: Strassen’s running time relative to the standard algorithm at the same n , as a function of the cut-off. The shaded band is the range predicted by H2; the control $n_0 = n$ should lie on the dashed line.

at $n = 512, 1024$ and 2048 , and $n_0 = 64$ is never more than 5% behind. The smallest cut-off behaves differently at each size. With $n_0 = 8$, Strassen’s algorithm is 16% slower than the standard one at $n = 512$, level with it at $n = 1024$ and 15% faster at $n = 2048$. H2 is supported: the optimum falls inside the predicted range and does not move with n .

Fitting over $n \geq 512$ gives a growth exponent of 3.037 ± 0.025 for the standard algorithm and 2.834 ± 0.004 for Strassen’s, a gap of 0.203 ± 0.025 . The local exponents in Figure 4 fill in the detail. For $n \geq 512$, Strassen’s stay between 2.82 and 2.85, always above $\log_2 7$. The standard algorithm’s are 2.93, 3.12 and 3.03 over the doublings from 512 to 4096. Below $n = 512$ both vary widely, from 2.00 to 3.84. H3 is supported. Both exponents land close to the predicted values, the gap is twice the minimum of 0.1 that H3 asked for, and the standard algorithm’s exponent sits slightly above 3.

Figure 5 plots $E(n)$ at $n_0 = 64$. It climbs from 1.7×10^{-15} at $n = 128$ to 1.1×10^{-13} at $n = 4096$, growing by a factor of 2.0 to 2.5 with each doubling, and a fit gives $E \propto n^{1.21 \pm 0.02}$.

Table 3: Median relative time $t_{\text{str}}/t_{\text{std}}$ by cut-off and size; the minimum of each column is in bold.

n_0	$n = 512$	$n = 1024$	$n = 2048$
8	1.156	1.002	0.851
16	0.800	0.676	0.602
32	0.733	0.617	0.549
64	0.747	0.633	0.557
128	0.809	0.699	0.603
256	0.896	0.740	0.651
512	0.959	0.802	0.697
1024	—	1.019	0.883
2048	—	—	1.003

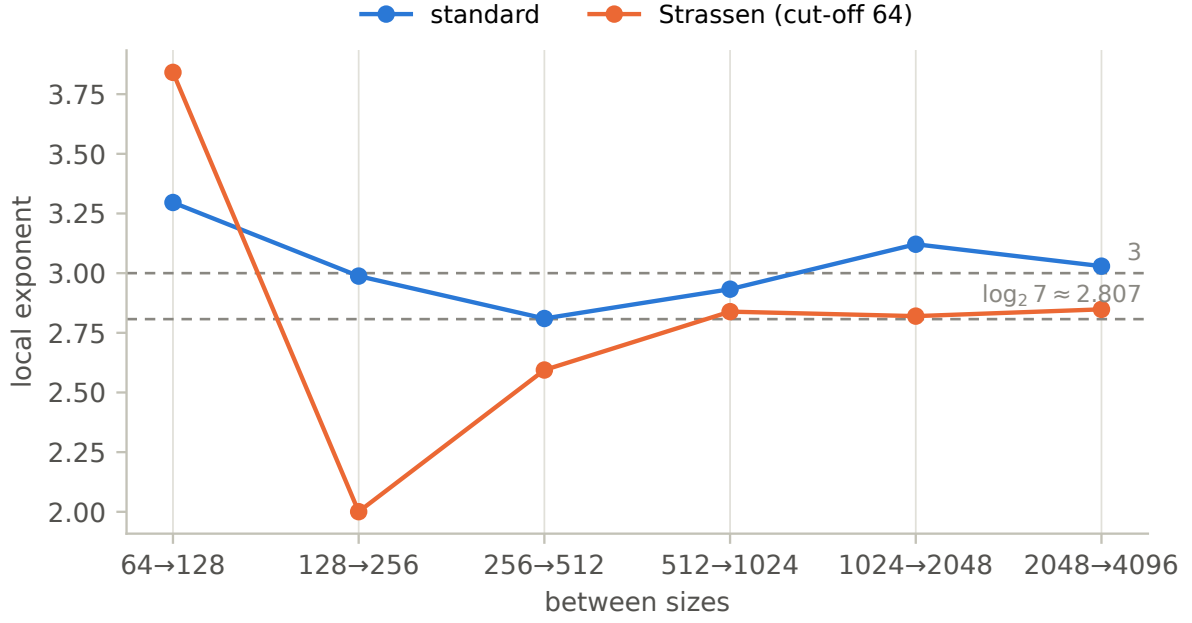


Figure 4: Local exponent $\log_2(t(2n)/t(n))$ between consecutive sizes, with the theoretical exponents 3 and $\log_2 7$ as dashed lines.

No repetition at $n_0 = 64$ went above 1.15×10^{-13} . The largest value anywhere in the study, 3.0×10^{-13} , came at $n = 2048$ with $n_0 = 8$, and Figure 6 shows the pattern behind it: at a fixed size, E rises steadily as the cut-off comes down. The bound in H4 holds with almost an order of magnitude to spare. The growth rate does not match, since E grows faster than the doubling H4 predicted, so H4 is only partly supported.

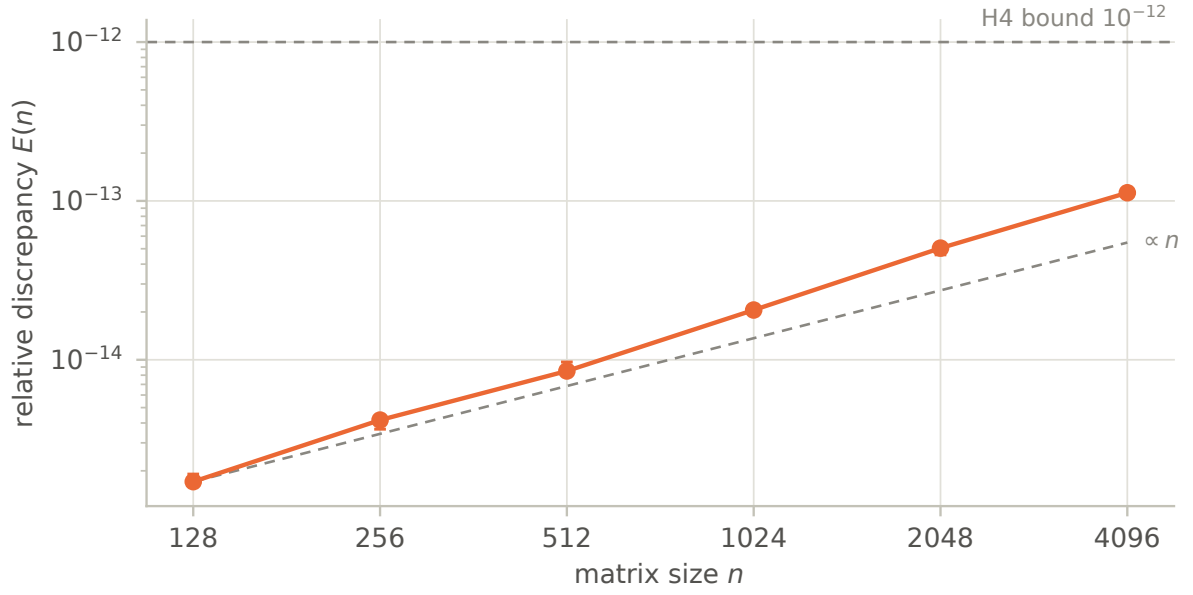


Figure 5: Relative discrepancy $E(n)$ at $n_0 = 64$, with a reference line proportional to n and the bound predicted by H4. At $n = 64$, $E = 0$ because no recursion takes place, so that size is omitted.

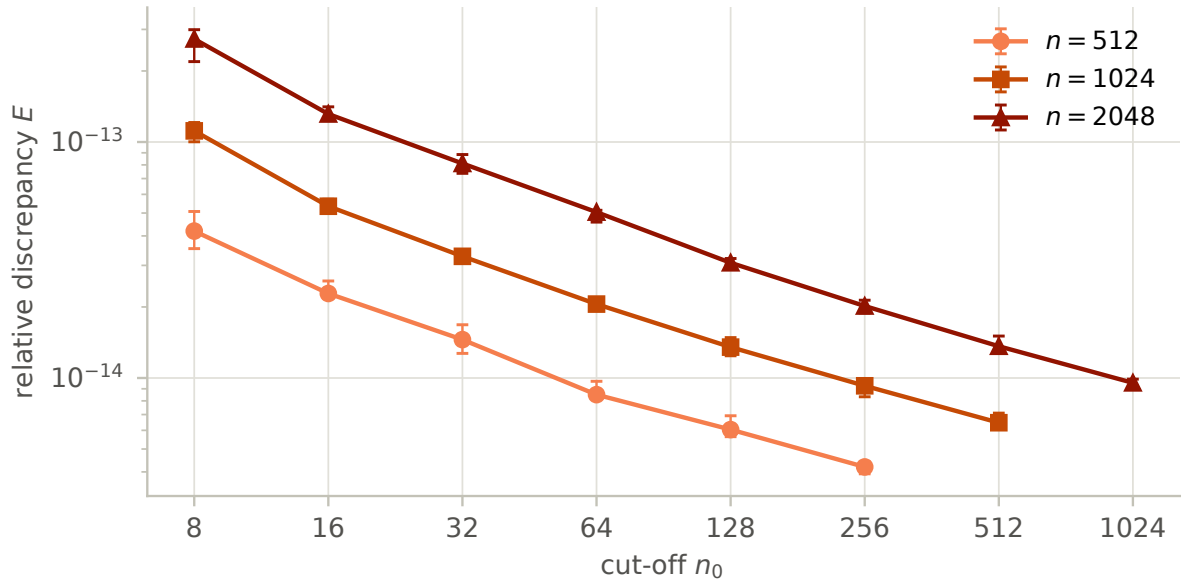


Figure 6: Relative discrepancy E as a function of the cut-off at the three largest sizes. Each halving of n_0 adds one level of recursion.

2.4 Analysis

Counting operations underestimated what Strassen’s algorithm costs. At $n = 128$, one level of recursion does about 11% fewer floating-point operations than the standard algorithm (Section 2.1), and yet it lost in every repetition. The extra arithmetic is not the problem: eighteen additions of 64×64 blocks come to about 7.4×10^4 operations, which is negligible. What the count leaves out is memory traffic. One level of this implementation allocates and zero-initialises 33 temporary half-size matrices (eight copied blocks, eighteen sums and differences, and seven products), and copies data into or out of most of them. The standard algorithm allocates only its result. Seven 64×64 products take about 2.1 ms at the measured speed, so of Strassen’s median of 4.9 ms at $n = 128$, roughly 2.8 ms goes on work the count never sees.

Things improve quickly as n grows. Each new level multiplies the saving by $8/7$, while the extra work at the top, proportional to n^2 , shrinks next to the $\Theta(n^{\log_2 7})$ multiplications underneath. If additions and copies were free, $L = \log_2(n/64)$ levels would give a speedup of $(8/7)^L$. The measured speedup reaches 85% of that at $n = 256$ and 90–93% from $n = 512$ upwards.

The crossover itself is shakier than it looks. In O2, lowering the cut-off from 128 to 64 splits every 128×128 block once more, which is exactly what made Strassen’s algorithm lose at $n = 128$. Inside the larger recursions the same step made it faster, in all fifteen paired comparisons, by 7–8% of the total time at each of the three sizes. That works out at about 0.2 ms saved per block. Done as a standalone multiplication, the same split cost about 2 ms more than the standard algorithm. So the cost of one level of recursion depends on where it is measured. The data do not show why. Costs paid once per top-level call, such as getting memory from the operating system, would hit the shortest measurements hardest, but this is only a conjecture. In practice the cut-off, which is measured inside the recursion, says more about when a split pays off than the standalone crossover does.

The best cut-off stays put as n changes because the choice it makes is local. Whether a block of size m is worth splitting once more comes down to one standard multiplication of size m against seven of size $m/2$ plus the block operations, and none of that depends on how big the original problem was. The data put the break-even point between 32 and 64. At every size, splitting the 64×64 blocks cuts the running time by about 2%, splitting the 32×32 blocks adds 9–10%, and splitting the 16×16 blocks adds 41–48%.

The same local rule accounts for both sides of the U. On the left, the losing splits happen at the bottom of the recursion, at the same block sizes whatever n is, and a larger n simply adds more winning levels on top. That is why $n_0 = 8$ is 16% slower than the standard algorithm at $n = 512$ and 15% faster at $n = 2048$. On the right, a single level of recursion ($n_0 = n/2$) already brings Strassen’s algorithm down to 0.90, 0.80 and 0.88 of the standard algorithm’s time, roughly the $7/8$ that removing one multiplication in eight allows. At $n = 1024$ it even dips below $7/8$, which saving multiplications cannot explain on its own; the value sits inside that configuration’s spread between repetitions (0.75 to 0.92).

Strassen’s exponent sits above $\log_2 7$ for the reason H3 gave. At a fixed cut-off, doubling n adds one level of recursion, so $t(2n) = 7t(n) + \Theta(n^2)$, and each local exponent overshoots $\log_2 7$ by $\log_2(1 + \Theta(n^2)/7t(n))$. The measured overshoot is between 0.01 and 0.04, so the block operations at the top level add only 1–3% to the time of the seven products below them. The local exponents do not settle towards $\log_2 7$ in a clean line, because they vary from one size to the next by about as much as this overshoot.

The standard algorithm needs a different explanation. A positive lower-order term would pull the fitted exponent of a $\Theta(n^3)$ algorithm down, not up, so the excess has to come from each operation getting more expensive as n grows, and that is what the data show. The standard algorithm’s time per n^3 is roughly flat at $n = 512$ and 1024 (1.20 and 1.15 ns), jumps by 9% to 1.25 ns at $n = 2048$ and stays there (1.27 ns) at 4096. Those are the local exponents of 3.12 and then 3.03. The **i-k-j** loop reads all of B once for every row of A , and B takes up $8n^2$

bytes: 8 MB at $n = 1024$, 32 MB at $n = 2048$. The jump is consistent with B no longer fitting in cache, so that every pass over it goes to main memory. Strassen’s base case always works on 64×64 blocks, whose three operands take 96 KB whatever n is, and its multiplications never cross that line. Below $n = 512$ the local exponents are erratic, partly from the noise in the shortest measurements and partly from the jump from no recursion to one level at $n = 128$, which is why the fit starts at 512.

The growth of E has two causes, and the cut-off sweep separates them. At a fixed cut-off, doubling n both makes every block bigger and adds a level of recursion. Comparing sizes at the same number of levels isolates the first effect: E grows by a factor of 1.45 per doubling (geometric mean of 13 comparisons, range 1.18–1.59). Comparing cut-offs at the same size isolates the second, and each extra level multiplies E by 1.60 (18 comparisons, range 1.41–2.09). Together they give 2.3, in line with the observed 2.0 to 2.5 per doubling. H4 only allowed for the first effect.

The per-level factor comes from the way Strassen’s algorithm is built. Its sums and differences of blocks, such as $A_{21} - A_{11}$, combine entries of opposite sign, so the element-wise bound of the standard algorithm,

$$|C - \hat{C}| \leq \gamma_n |A| |B|, \quad \gamma_n = \frac{nu}{1 - nu}, \quad (10)$$

where u is the unit roundoff, no longer applies. Only a norm-wise bound holds, and its constant grows geometrically with the number of levels.¹ The observed factor of 1.6 per level is far below that worst case. With the nonnegative test matrices, $|A||B| = |AB|$, so (10) guarantees the standard algorithm a small relative error in every single entry, the most favourable situation it could have (Section 2.5). In absolute terms both algorithms stay accurate: E at $n = 4096$ is about 10^3 units of roundoff ($u = 2^{-53} \approx 1.1 \times 10^{-16}$).

The cut-off, in other words, trades speed against accuracy. At $n = 2048$, lowering it from 64 to 32 saves 1.4% of the running time and multiplies E by 1.6.

2.5 Threats to Validity

The biggest caveat is the test data. With entries drawn from $[0, 1)$, every term the standard algorithm adds is nonnegative, so it never cancels, and by (10) it keeps a small relative error in every entry. Strassen’s differences of blocks do cancel, so the accuracy comparison is made in the case most favourable to the standard algorithm. Because all entries of the product are close to $n/4$, E also gives a fair picture of the error in each individual entry. With badly scaled matrices it would be dominated by the largest entries and could hide large errors in the small ones, and with entries of both signs both algorithms would lose accuracy to cancellation. Repeating O4 with entries in $[-1, 1)$, and with rows and columns scaled over several orders of magnitude, would show how far the results carry over. Real values were needed all the same: with small integers both algorithms would be exact and E would be zero.

E itself, defined in (9), compares two inexact results rather than measuring either against the exact product, because no higher-precision reference was computed.

Timings below a few milliseconds are unreliable. At $n = 64$ the control ratio came out at 1.17 although both algorithms run the same code, so the conclusions at $n \leq 128$ rest on all repetitions agreeing, not on the size of the differences. The context dependence discussed in Section 2.4 points the same way: a standalone timing of a small multiplication may not reflect its cost inside a larger one. Timing many consecutive multiplications per measurement would have reduced the noise.

The samples are small, with five repetitions (three at $n = 4096$) and only four sizes in each exponent fit, so the standard errors of the fits are rough. Pairing the measurements protects

¹N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, 2002, Ch. 23: in the max norm used here, the bound for Strassen’s method with cut-off n_0 is proportional to $(n/n_0)^{\log_2 12}$, a factor of up to 12 per level.

the ratios against slowdowns that hit a whole repetition, such as the 12% slowdown of the fourth repetition at $n = 2048$, whose control ratio was still 1.00, but not against a slowdown in the middle of a single measurement. Finally, everything here describes one implementation, compiled by one compiler on one machine. Working on views of the matrices instead of copies would cut the overhead per level and probably move the crossover and the best cut-off down, while a tuned BLAS routine for the standard algorithm would move them up.

3 Conclusions and Reflections

3.1 Unexpected Findings

What surprised me most was how much a single level of recursion costs. With a cut-off of 64, Strassen’s algorithm runs exactly the same code as the standard one at $n = 64$, so $n = 128$ is the first size where it really recurses. By the operation count, one level should already win there, since it needs about 11% fewer floating-point operations. H1 gave a more cautious range, but I still expected at least a small gain. It lost in every repetition, taking 1.39 times as long. That one level added roughly 2.8 ms, almost as much as the whole standard multiplication at that size (2.9 ms), and the median speedup did not cross 1 until around $n = 210$.

The local exponents tell the same story. From $n = 64$ to 128, Strassen’s time grows fourteen-fold, an exponent of 3.84 against 3.30 for the standard algorithm. One doubling later it swings the other way, to 2.00, well below $\log_2 7$, and only from $n = 512$ on does it settle between 2.82 and 2.85.

The second finding grew out of the first. Lowering the cut-off from 128 to 64 in the sweep splits every 128×128 block once more, the very operation that lost at $n = 128$. Inside the larger recursions that split paid off. It made Strassen’s algorithm faster in all fifteen paired comparisons, saving about 0.2 ms per block, while on its own the same split had cost about 2 ms extra. The cost of a level depends on where it is measured, and a standalone crossover understates how early Strassen’s algorithm helps inside a bigger computation. I still do not know why. It did change which measurement I trust, though: the cut-off, measured inside the recursion, is the better guide.

The accuracy results held the third surprise. H4 assumed the error grows because n grows. At a fixed cut-off, doubling n actually changes two things, since the blocks get bigger and the recursion gets one level deeper, and the cut-off sweep let me pull those apart. Depth mattered more. Each extra level multiplies E by about 1.6 at every size tested, whereas doubling n at a fixed depth multiplies it by about 1.45. The fastest setting is not the most accurate one, either. At $n = 2048$, the fastest cut-off, 32, beat 64 by only 1.4% and produced 1.6 times the error.

3.2 Initial Misconceptions

My first draft of H3 said that both measured exponents would come out above their theoretical values, because the $\Theta(n^2)$ overheads were not yet negligible. That holds for Strassen’s algorithm, where the overhead enters the recurrence $t(2n) = 7t(n) + \Theta(n^2)$. For the standard algorithm it is simply wrong: adding a positive $\Theta(n^2)$ term to a $\Theta(n^3)$ running time pulls the fitted exponent down, not up.

The error came to light while I was reviewing the hypotheses with Claude (Section 4), before any experiment had been run. I rewrote H3 so that a standard exponent above 3 would be put down to cache effects, and the results bore that version out. Over the sizes used for the fit, the standard algorithm’s local exponent passed 3 only from $n = 1024$ upwards, consistent with B no longer fitting in cache.

I also assumed that timing a single call would be precise enough at every size. For the smallest sizes it was not. At $n = 64$ both algorithms run identical code, yet single measurements of that computation ranged from 0.26 to 0.58 ms, and the control ratio came out at 1.17 instead

of 1. Below a few milliseconds the noise is as large as the effects being measured. At $n \leq 128$, my conclusions rest on all five repetitions agreeing, not on the size of the differences. Next time I would time many consecutive multiplications and divide by their number.

The entries in $[0, 1)$ were a choice of convenience, and I did not think about what it implied. Only while interpreting O4 did I see that nonnegative matrices are the most favourable case there is for the accuracy of the standard algorithm. The timings do not depend on the values of the entries. The accuracy results very much do.

3.3 Key Insights

The biggest lesson is that an asymptotic advantage only counts above a certain block size, and that size is set by constant factors and memory behaviour rather than by the exponent. Strassen’s algorithm is not a replacement for the standard one. It works on top of it, as a hybrid that stops recursing at the break-even point. In my implementation that point lies between 32 and 64 at every size, and even above it progress is gradual: it took six levels of recursion, at $n = 4096$, to halve the running time. What limits it is memory traffic, not arithmetic. The same applies to the measured exponents, which describe an implementation running on a memory hierarchy as much as an algorithm; the standard algorithm’s local exponent of 3.12 comes from how its memory accesses meet the cache, not from the triple loop itself.

Speed and accuracy turned out to be controlled by the same parameter. Every level of recursion saves one multiplication in eight and multiplies the error by about 1.6, so choosing the cut-off means choosing a point on that trade-off. Behind it is a weaker guarantee: Strassen’s algorithm gives up the element-wise error bound of the standard algorithm for a norm-wise one. How much that matters depends on the data, and my nonnegative test matrices were the kindest case possible.

The design of the measurements decided what I could conclude. The control runs, with ratios of 0.96, 1.02 and 1.00, showed that the timing could be trusted. Computing every ratio within a repetition absorbed a slowdown of 12% that hit one whole repetition at $n = 2048$. Fixing the analysis rules before looking at the data kept me from picking the fitting range or the definition of the crossover to suit the results. The same design also showed its limit at the smallest sizes, where a single timed call was not precise enough.

So, is Strassen’s algorithm worth using, and when? For this implementation, yes, within limits:

- From about $n = 256$ upwards. That is the first size at which it won in every repetition, and its advantage grows steadily from there, down to half the standard algorithm’s running time at $n = 4096$.
- With a cut-off of 32 to 64. Below that, splitting a block costs more than it saves.
- When a small loss of accuracy is acceptable. The relative discrepancy stayed below 1.2×10^{-13} at the cut-off of 64 and was at most 3.0×10^{-13} in any configuration.
- On well-scaled data. My test matrices were the case most favourable to the standard algorithm; for badly scaled data, where a norm-wise guarantee says little about the small entries, the standard algorithm remains the safer choice.

4 Use of AI Tools

Tools and extent. I used Claude (Anthropic) heavily throughout the project and Gemini (Google) as a second opinion. Claude wrote most of the measurement and plotting code and reviewed the code I wrote. Gemini produced one rewrite of Section 2.1, and I also used it to contrast its

answers with Claude’s, so that a claim one tool had made up was less likely to go unnoticed. Every AI-assisted part of the code is marked with a comment in the source files.

What I used them for. I wrote the two algorithms and the helper functions in `matrix.hpp` myself. Claude reviewed them and suggested corrections that I checked and typed in: making the cut-off a run-time parameter and rejecting values below 1, passing a seeded generator to `fill_random` so that every matrix can be reproduced, adding a missing header, and fixing my `.gitignore`. It proposed edge cases and controls for the tests, among them the cut-off $n_0 = n$, which has to match the standard algorithm, and a tolerance check on every result. It wrote the experiment harness in `main.cpp` and the analysis script `plot_graph.py` to a structure we agreed on. I compiled and ran everything on my own machine, and every number in this report comes from those runs. I also used Claude to understand the theory behind the experiments: the recurrences for the running time, what a local exponent measures, and why the two algorithms have different error bounds. For the report, it computed derived quantities such as the ideal speedup $(8/7)^L$, drafted the text of Sections 2.2 to 3 from my results and our discussions, which I then edited, and revised their wording at my request. Claude also wrote Section 2.5; I reviewed it afterwards and had it condensed and revised where I considered it necessary. The decisions stayed with me. I chose among the options it proposed, picked the findings in Section 3 from my own experience, and cut material I considered out of scope.

Where they were wrong. Four cases stood out.

- Gemini’s rewrite of Section 2.1 read well but misdescribed my own work. It claimed that every hypothesis had been formulated before any measurement, although H4 drew on a preliminary check; it defined the error measure with the Frobenius norm, while the program records the max norm; and it assigned the predicted exponents to the wrong algorithms. Claude found these problems, I checked each one against the code and the data, and the final version keeps the rewrite only with the corrections. Comparing the two tools is what exposed the errors, even if this time they were Gemini’s.
- The first version of the plotting script declared a crossover wherever the median speedup exceeded 1. On test data it reported a win at a median of 1.001 even though some repetitions went the other way, so it was measuring noise rather than a crossover. Claude caught this while testing the script, and I adopted the stricter rule that Strassen’s algorithm must be faster in every repetition (Section 2.2).
- Claude reported one figure wrongly in our discussion, saying that Strassen’s algorithm was 28% slower at $n = 128$ when the data give 39%; it had converted the speedup incorrectly. It corrected this when it recomputed the values from the CSV, and from then on every number in the report was recomputed from the data rather than copied from the conversation.
- The drafts left out the most important simplification of the study, the choice of entries in $[0, 1)$. I noticed the gap and asked for it to be discussed, and it turned out to be the main limitation of the accuracy results (Section 2.5). An earlier draft of the methodology also reported results, which I had removed.

What I understood and what I took on trust. I understand both algorithms, the structure of the harness, why only powers of two give distinct cut-offs, why times are paired within a repetition, and how each objective is read from the table of results. I can follow the reasoning behind each conclusion in Section 2.4. For `plot_graph.py`, which Claude wrote, I made an effort to understand it thoroughly after the analysis was finished, going through it function by function, including the standard-error formula used in the exponent fits. I now know what each function does and where it fits in the script, but not what every individual line does, particularly in the matplotlib formatting. I still take one thing on trust: the exact form of Higham’s error bound for Strassen’s method, in particular the factor $(n/n_0)^{\log_2 12}$, comes from Claude, and I

have not checked it against the book. The explanation for why a split costs more as a standalone multiplication than inside the recursion is only a conjecture, as Section 2.4 says, and my data cannot confirm it.